# Soup DevOps: release and training contracts

Maksim | 11 November 2026 | Two-page evidence note

A passing source test does not establish that an installed package works, or that its backward pass trains the intended weights. This exercise retains a published greeting CLI as the release baseline and adds an independently written LoRA/Triton fixture relevant to Soup's post-training work. Private repository.

## Original release failures, demonstrated in Actions

**python310.** Import datetime.UTC while claiming Python 3.10 support: both 3.10 jobs fail. Naive GREEN / Installed RED.

**windows.** Import Unix-only pwd: all five Windows jobs fail. Naive GREEN / Installed RED.

**missing-data.** Omit greeting.txt: installed wheel, rebuilt sdist and Docker fail. Naive GREEN / Installed RED.

**entrypoint.** Point the script at missing_main: source main() passes, installed CLI fails. Naive GREEN / Installed RED.

The corrected baseline passes Ubuntu/Windows x Python 3.10-3.14, testing the wheel and independently rebuilt sdist in isolated runtimes. Docker executes the same wheel non-root, offline and read-only. Corrected matrix; OIDC publication and index verification; TestPyPI 0.9.3. Its immutable tag and published bytes are preserved.

## Training contracts: why this is useful for Soup

Soup's engineering story describes a gradient bug that survived releases. A decreasing loss or correct forward value is therefore insufficient. The new fixture compares custom SwiGLU forward/backward with native autograd, adds FP64 finite-difference gradcheck, and compares two LoRA SGD steps with independent functional.linear projections. Frozen weights must remain exact; all A/B gradients and updates must match. Zero first-step A gradients are valid when B starts at zero, but absent gradients fail.

Four injected faults each pass naive CPU source forward: a missing derivative term, detached LoRA A, omitted kernel-contract.json, and a Triton tail-mask error. Strong installed wheel and rebuilt-sdist checks reject every fault for its specific cause. Local execution verified all **17 comparisons**; arbitrary build/import errors cannot satisfy a negative case. Original failure logs and distribution bytes are retained.

Two pinned stacks passed local CPU and real Triton-interpreter checks: **PyTorch 2.6.0+cpu / Triton 3.2.0** and **PyTorch 2.14.0+cpu / Triton 3.8.0**. CPU checks include FP32/FP16/BF16, empty/strided inputs and irregular tails; interpretation is FP32 only. Release CI gates on six Linux Python 3.11-3.13/stack combinations and two Windows Python 3.12/stack combinations, plus bare CLI checks without ML imports. Current CI results; Contracts and reproducible commands.

# Trust, GPU boundary and release verdict

## What the pipeline can establish

**Artifact identity:** one build creates a wheel from sdist. Commit, run ID, attempt, distribution digests and profile hashes are bound in a manifest. Consumers verify it; sdists are also rebuilt independently. Installed probes run outside the checkout with isolated, optimized Python. Changed dependencies or incomplete numerical reports fail. No restored environment cache or editable source installation is used.

**Dependency provenance:** ML CPU wheels use reviewed official PyTorch URLs/SHA256s; Triton and transitive tools use hashed locks. Downloads are verified before offline installation, avoiding a mixed extra index and multi-GB CUDA downloads. Exact inventory and pip check verify the selected closure. Setuptools is explicit where Torch requires it. Historical HTTPX/Tenacity API and undeclared-import experiments remain available separately.

**Privileges:** PR jobs are read-only, with pinned Actions and no persisted checkout credentials or publisher secrets. Only a matching tag on main can publish the tested bytes through OIDC; the training workflow is now a required dependency. Registry verification checks served hashes and a clean install after publication. Corruption fails immediately; transient registry availability receives bounded retries.

## Native GPU: implemented probe, unverified execution

The separate fixture implements --backend cuda for NVIDIA SM80+: real device discovery, Triton forward/backward, numerical/LoRA contracts and synchronization before success. Interpreter mode cannot record native CUDA success. CPU workers were checked to fail with gpu-unavailable, exit 3 and native_cuda=false. **No native GPU run, compiled-kernel correctness, sanitizer result, memory saving or speedup is claimed.** Pin a separate CUDA/driver/image profile and execute installed artifacts there before advertising a GPU gate.

## GPU budget proposal

Runpod Secure Cloud RTX 4090: $0.89/hour, checked 10 November 2026. Thirty 10-minute nightlies, four 20-minute releases and two minutes provisioning/teardown per run total 7.47 hours: $6.65 compute; prorated disposable 20 GB disk about $0.02; with 25% allowance, **$8.33/month**. Excludes tax, egress and GitHub minutes. Price source. No paid GPU was rented.

A proposed broker validates GitHub OIDC, holds provider credentials outside workloads, reserves worst-case lease cost, caps concurrency/rate and enforces a $10 monthly budget. An external watchdog terminates Pods and volumes at 15/30-minute limits, including after Actions cancellation. These controls are designed, not deployed.

## Verdict and remaining concern

TestPyPI publication is demonstrated; unattended real PyPI is not authorized. Private-repo branch/tag protection currently returns HTTP 403 on this account plan. Require protection and review of workflow/lock changes first. This training fixture is not Soup's production code or full dependency stack; CUDA, macOS, ARM and unselected versions remain untested. Pinned hashes do not prove benign code. A compromised maintainer can change both code and checks; post-upload verification cannot recall installed or mirrored files. Yank a faulty release and publish a new version.

## Surprise and AI disclosure

The strongest surprise was that a correct forward pass can coexist with broken training: zero-initialized LoRA B also makes first-step A gradients an easy false alarm. Codex drafted and executed the automation; local review corrected YAML quoting and strengthened the independent oracle and evidence checks. Claude Code and subagents were not used. The applicant configured the TestPyPI Trusted Publisher and independently created a fresh local venv, installed version 0.9.3 from TestPyPI, and ran the installed CLI. The observed JSON contained Hello, Soup! and version 0.9.3. The broader matrix and negative tests were automated; no manual review of every Actions log is claimed. New training and compatibility checks were AI-assisted; no new applicant-independent verification or publication is claimed.